

12

CI/CD at Scale in GKE

Google Cloud

Today, many organizations are modernizing their applications. Application modernization enables businesses to leverage fully-managed, scalable infrastructure, enterprise features, and enhanced security.

The modernization process is often complex and can slow innovation. That's where CI/CD comes in, providing developers with tools and automation to achieve outcomes repeatedly and at scale.

CI/CD at scale in GKE

01	CI/CD in Google Cloud
02	Cloud Build and GKE
03	Cloud Deploy and GKE
04	Cloud Deploy: Policies, Deployments and Security
05	Cloud Run and Knative Serving
06	Cloud Deploy and Knative Serving
07	CI/CD in a private network
08	Securing the software supply chain



In this section, titled CI/CD at scale in GKE, you learn to:

- Describe the core components of Google Cloud's CI/CD pipeline and how they address common challenges in application modernization.
- Analyze how Cloud Deploy integrates with GKE to manage Kubernetes manifests and control deployments.
- Compare and contrast the deployment strategies for Knative Serving within GKE.
- Explain the steps required to establish a peered VPC connection for secure CI/CD in a private network.
- Evaluate the various security measures and tools available within Google Cloud for securing the software supply chain.

CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network
- 08 Securing the software supply chain



Google Cloud

Let's start by introducing CI/CD in Google Cloud.

Application modernization consists of modernizing the application stack, infrastructure, and processes, and shifting to a fully-managed, scalable infrastructure.

CI/CD operationalizes the process and provides developers with tools and automation to achieve application modernization outcomes repeatedly and at scale.

App modernization is typically a gradual process. Let's explore how it fits your existing DevOps toolchain.

Accelerate State of DevOps report

Software delivery performance metric	Elite	High	Medium	Low
Development frequency	On-demand (multiple per day)	Daily to weekly	Weekly to monthly	Less often than every 6 months
Lead time for changes	< 1 hour	1 day to 1 week	1 week to 1 month	1 month to 6 months
Time to restore service	< 1 hour	< 1 day	< 1 day	1 week < > 1 month
Change failure rate	5%	20%	10%	40%

Google Cloud

The Accelerate State of DevOps report highlights examples of CI/CD excellence. To build the report, the Google Cloud DevOps Research and Assessment (DORA) team gathered feedback from 39,000 professionals worldwide.

Teams are classified as elite, high, medium, or low performers based on their software delivery. Evaluation factors include deployment frequency, lead time for changes, mean-time-to-restore, and change fail rate.

In 2024, elite performers continued to accelerate their software delivery. These performers increased lead time for changes from less than one day to less than one hour.

The data shows that elite performers deploy 192x more frequently than low performers, have a 127x faster lead time to deploy, a 8x lower change failure rate, and an impressive 2293x faster time to recover from incidents when failure does happen.

Challenges in the CI/CD journey



Scale

- Infrastructure costs
- Environment variability
- Visibility at scale
- Onboarding velocity



Security

- Separation of roles
- Release promotion controls
- Reliable rollbacks
- Auditability and traceability



Integration

- Increased coordination and dependencies
- Slower time to recovery and increase the complexity of root cause analysis

Scale, security, and integration are common challenges encountered in the CI/CD journey.

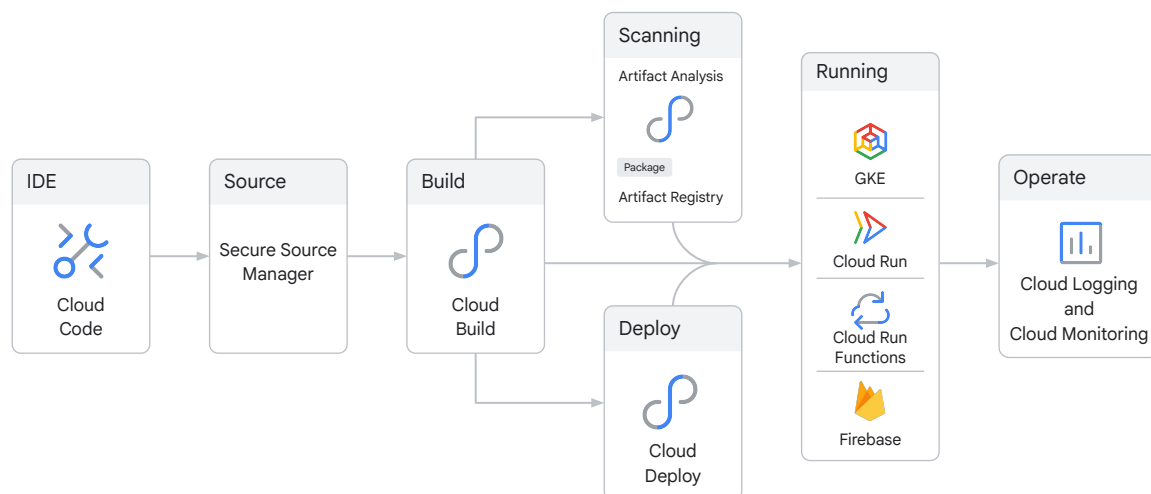
Scaling infrastructure can be costly. Creating fixed replicas of production, staging, and development environments can be expensive. Variations between target environments result in divergence and increased pipeline complexity.

Visibility (health and activity monitoring) and onboarding also becomes difficult as your environment configuration increases in complexity.

Another frequently encountered challenge is security. In some cases you may need to split access based on roles to meet compliance standards. In these scenarios a single CI/CD tool is insufficient. Release promotion controls, reliable rollbacks, and traceability are other issues.

And integrating operations teams with logging and monitoring can lead to increased coordination and dependencies. These increases result in a slower time to recovery and increase the complexity of root cause analysis.

DevOps and CI/CD in Google Cloud



To help solve these challenges, Google Cloud offers a complete, fully-managed, serverless, DevOps pipeline with modular systems that can be replaced with existing third-party tooling if desired.

Let's explore the pipeline's individual components.

- Cloud Code is an IDE plugin similar to VS Code and IntelliJ. You can use Cloud Code to write, debug, and deploy Kubernetes applications. It features pre-configured samples for fast deployments.
- Secure Source Manager is a regionally deployed, single tenant, managed **source code repository** hosted on Google Cloud that allows you to store your code and configure triggers to your CI/CD pipelines.
- Cloud Build is a serverless CI/CD platform for building, testing, and deploying applications in the cloud at scale.
- Artifact Registry stores, manages, scans, and secures your container images and packages.
- Cloud Deploy is a declarative continuous delivery system.
- And you can use Cloud Logging and Cloud Monitoring to observe application performance after code is in production.

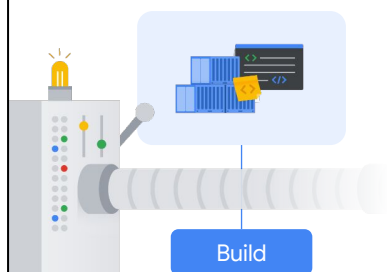
CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network
- 08 Securing the software supply chain

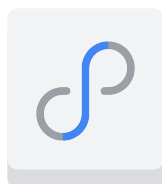


In a CI/CD pipeline, the *build* stage refers to when raw source code is transformed into a deployable package.

Cloud Build



When raw source code is transformed into a deployable package.



Cloud Build

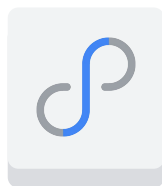
Can execute builds on:

- Google Cloud infrastructure.
- An on-premises environment.
- A combination of the two.

The process typically involves fetching dependencies, compiling the code, and packaging the application.

To help with this process, Google offers Cloud Build, which is a service that can execute builds on Google Cloud infrastructure, an on-premises environment, or a combination of the two.

Cloud Build



Cloud Build

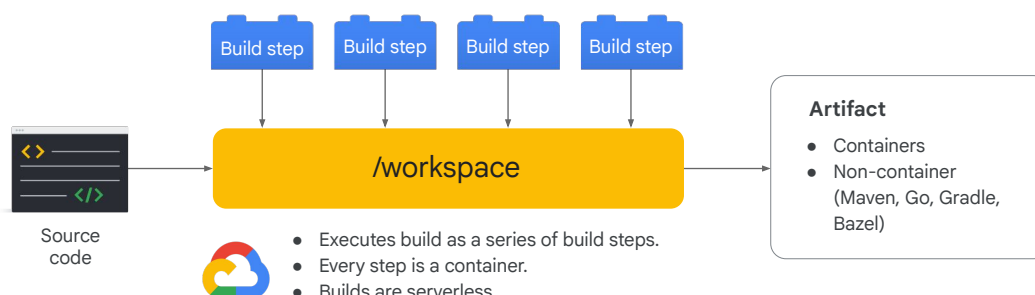
- ✓ Can import source code from a variety of repositories or cloud storage space.
- ✓ Can execute a build to defined specifications.
- ✓ Can produce artifacts, such as container images.
- ✓ Can automatically start the build process in response to specific changes in your code.
- ✓ Can be used to execute code provided by solutions like Terraform and Helm.

Cloud Build can import source code from a variety of repositories or cloud storage spaces, execute a build to defined specifications, and produce artifacts, such as container images.

Cloud Build can be customized to automatically start the build process in response to specific changes in your code. This automation is achieved through pre-defined triggers, which define the specific events that will initiate a build.

Cloud Build can also be used to execute the code provided by solutions like Terraform and Helm, and can orchestrate many types of services, from data to IoT pipelines.

The build process



The build process can happen sequentially or in parallel.

The build process starts by mounting a common volume named `/workspace`. Then, the source code of an application is added to this volume.

Cloud Build creates each of the build steps as independent containers. This allows each content to operate on the common workspace. This environment reduces the need for redundant data stores.

The build process is typically triggered by a change in your code repository. This change produces artifacts like containers or binaries.

Cloud builders

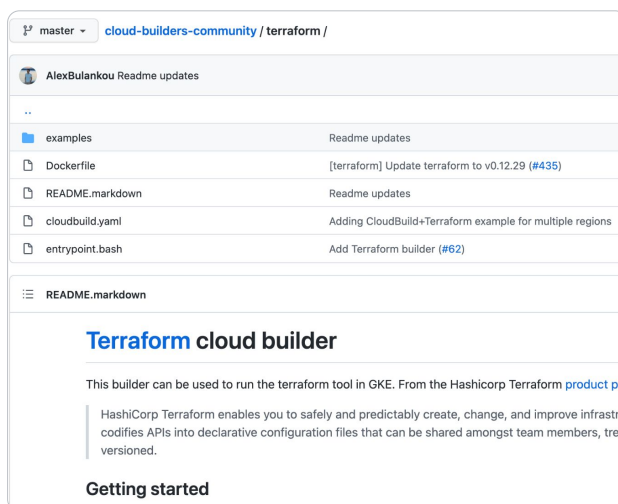
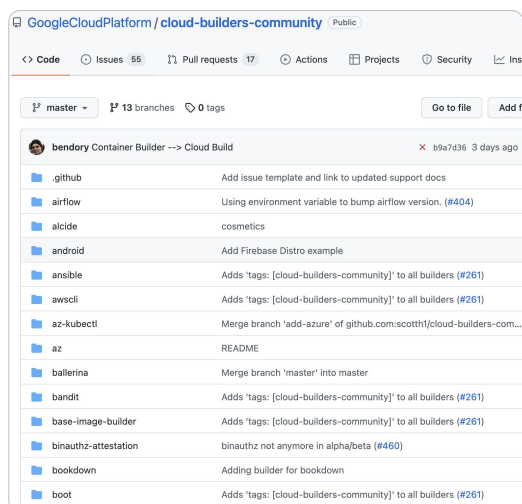
Containers with common languages and tools that you can use in build steps:

docker	gcr.io/cloud-builders/docker	docker example
go	gcr.io/cloud-builders/go	go example
gcloud	gcr.io/cloud-builders/gcloud	gcloud example
gradle	gcr.io/cloud-builders/gradle	gradle example
maven	gcr.io/cloud-builders/mvn	maven example
kubectl	gcr.io/cloud-builders/kubectl	kubectl example
npm	gcr.io/cloud-builders/npm	npm example

And many more: <https://github.com/GoogleCloudPlatform/cloud-builders>

Any container can be used as a build container in Cloud Build. Google provides pre-built containers that do not need modification.

Use community-contributed builders in your project



Or you can choose from a curated repository of containers created by the Google Cloud community. These containers are available at no-cost.

Specify build steps in a cloudbuild.yaml file

```
steps:  
- name: 'gcr.io/cloud-builders/npm'  
  args: ['install']  
- name: 'gcr.io/cloud-builders/npm'  
  args: ['test']  
- name: 'gcr.io/cloud-builders/docker'  
  args: ['build', '-t', 'us-central1-docker.pkg.dev/$PROJECT_ID/helloworld', '.']  
- name: 'gcr.io/cloud-builders/docker'  
  args: ['push', 'us-central1-docker.pkg.dev/$PROJECT_ID/helloworld']
```

Build steps are outlined in a configuration YAML file. In this example, an NPM container installs and tests the application and a Docker container creates the application container and adds it to the Container Registry.

Cloud Build creates the pipeline and executes the container

The screenshot displays the Google Cloud Build console for a successful build. The build ID is 010b976e, and it started on Oct 6, 2021, at 2:21:44 PM. The build was triggered by 'events-internal' from the source 'hudomju/events-app-internal' on the 'master' branch, with commit '6c8bd57'.

Steps	Duration	Build Log	Execution Details	Build Artifacts
Build Summary 6 Steps	00:02:23	☐ Wrap lines ☐ Show newest entries first ↓ EXPAND VIEW RAW		
0: gcr.io/cloud-builders/git clone https://github.com/hudomju/events-app-int...	00:00:01	18 BUILD 19 Starting Step #0 20 Step #0: Already have image (with digest): gcr.io/cloud-builders/git 21 Step #0: Cloning into 'events-app-internal' ... 22 Finished Step #0 23 Starting Step #1		
1: gcr.io/cloud-builders/npm install	00:00:10	24 Step #1: Already have image (with digest): gcr.io/cloud-builders/npm 25 Step #1: npm WARN read-shrinkwrap This version of npm is compatible with lockfileVersion@1, but package-lock.json was generated for lockfileVersion 26 Step #1: npm WARN server@1.0.0 No repository field. 27 Step #1: npm WARN optional SKIPPING OPTIONAL DEPENDENCY: fsevents@2.1.3 (node_modules/fsevents): 28 Step #1: npm WARN notsup SKIPPING OPTIONAL DEPENDENCY: Unsupported platform for fsevents@2.1.3: wanted {"os":"darwin","arch":"any"} (current: {"os" 29 Step #1: 30 Step #1: added 381 packages from 198 contributors and audited 382 packages in 7.284s 31 Step #1: 32 Step #1: 18 packages are looking for funding		
2: gcr.io/cloud-builders/npm test	00:00:02			
3: gcr.io/cloud-builders/docker build -t eu.gcr.io/justo-1510761744000/events-ap...	00:00:24			
4: gcr.io/cloud-builders/docker push eu.gcr.io/justo-1510761744000/events-app...	00:00:08			

After you submit the YAML file, Cloud Build creates the pipeline and executes the container so that it can be run in the cloud.

Cloud Build pushes a container to Artifact Registry

```
steps:  
- name: 'gcr.io/cloud-builders/npm'  
  args: ['install']  
- name: 'gcr.io/cloud-builders/npm'  
  args: ['test']  
- name: 'gcr.io/cloud-builders/docker'  
  args: ['build', '-t', 'us-central1-docker.pkg.dev/$PROJECT_ID/helloworld', '.']  
- name: 'gcr.io/cloud-builders/docker'  
  args: ['push', 'us-central1-docker.pkg.dev/$PROJECT_ID/helloworld']
```

Artifact Registry securely stores and analyzes your containers.

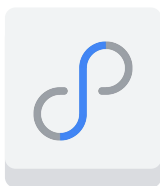
The container is then added to Artifact Registry, a fully-managed, secure, and scalable artifact repository for storing, managing, and distributing build artifacts.

Use Cloud Build to trigger a Cloud Deploy release

```
steps:
- name: 'gcr.io/cloud-builders/npm'
  args: ['install']
- name: 'gcr.io/cloud-builders/npm'
  args: ['test']
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-central1-docker.pkg.dev/$PROJECT_ID/helloworld', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'us-central1-docker.pkg.dev/$PROJECT_ID/helloworld']
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  entrypoint: gcloud
  args: [
    "beta", "deploy", "releases", "create", "rel-\\${SHORT_SHA}",
    "--delivery-pipeline", "helloworld-pipeline",
    "--region", "us-central1",
    "--annotations", "commitId=\\${REVISION_ID}",
    "--images", "helloworld=us-central1-docker.pkg.dev/$PROJECT_ID/helloworld"
  ]
```

For more advanced continuous deployment pipelines, you can trigger a Cloud Deploy release.

Cloud Deploy



Cloud Deploy



Provides a single place to view and manage all of your deployments.



Can be integrated with other services like Cloud Monitoring and Cloud Logging.



Deploy applications and containers from a variety of sources like GitHub, Bitbucket, Docker, Artifact Registry, and Helm.



Supports different deployment strategies.

Cloud Deploy provides a single place to view and manage all of your deployments, and can be integrated with other Google Cloud services, such as Cloud Monitoring and Cloud Logging.

It can deploy applications from many sources, including source code repositories, such as GitHub and Bitbucket; container images, such as Docker and Google Artifact Registry, and Helm charts.

And Cloud Deploy also supports many deployment methods. The list includes blue/green deployments, rolling updates, and Canary deployments.

Cloud Deploy is a powerful and easy-to-use tool that can help you improve the reliability, scalability, and security of Kubernetes deployments.

CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE**
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network
- 08 Securing the software supply chain



Cloud Deploy is a fully-managed, scalable Google Cloud service used to automate the deployment of applications to Google Kubernetes Engine. In this section, let's explore the tool in detail.

Cloud Deploy



Reduced overhead

- Fully-managed service
- Serverless



Enhanced control

- Environments and progressions
- Manual release approval
- Rollbacks made easy



Increased visibility

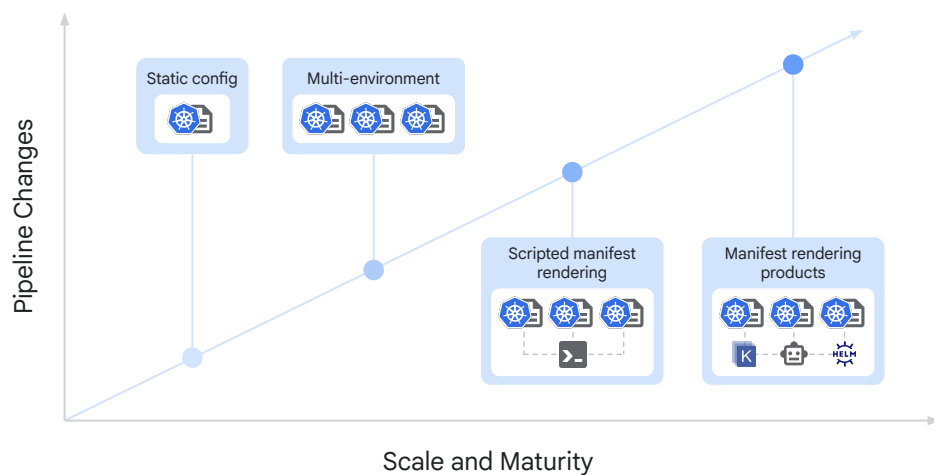
- Deployment status and metrics
- Objective-based SRE and DevOps
- Traceability and audit insights

Cloud Deploy is fully-managed and serverless, reducing operational overhead.

It provides enhanced control over the deployment lifecycle. Environments and progressions are consistent, and deployments are standardized and predictable. Manual release approvals can be implemented to add a layer of governance and verification. When encountering deployment issues, teams can use rollbacks to quickly revert to a stable state.

Finally, Cloud Deploy increases visibility into the deployment process. Developers can use integrated dashboards and logging to monitor performance and identify potential bottlenecks. The data can be used to drive objective-based SRE and DevOps processes, promoting a data-driven approach to continuous improvement. Traceability and audit insights secure deployment actions and ensure compliance.

Kubernetes is a journey

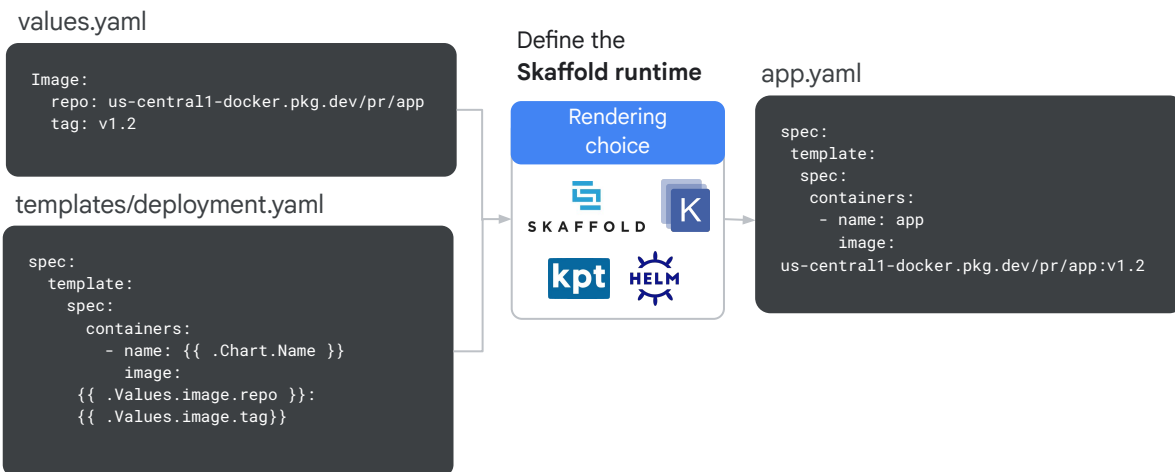


As organizations grow and environments diversify, managing DevOps permissions becomes complex and challenging.

Many organizations use custom scripting to render manifests for each environment, which leads to unreliable deployment pipelines.

As Kubernetes scale and maturity advances, many organizations adopt Kubernetes-native manifest rendering tools like Helm, Kustomize, and Kpt.

Cloud Deploy uses Skaffold to render Kubernetes-native applications



Google Cloud

Skaffold separates rendering tools from the delivery pipeline. This separation enables developers to use a rendering tool of their choice without impacting the delivery pipeline.

The most commonly used tools for packaging, customizing, validating, and applying Kubernetes resources are kpt (pronounced “kept”), Kustomize, and Helm, which Skaffold uses to render the Kubernetes manifests.

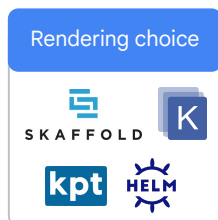
This is an example of a Helm package, with a values.yaml and a deployment.yaml file

The values depend on the environment that you are deploying to, like production and deployment.

Skaffold uses profiles to match the configuration with the deployment target so that a development Kubernetes cluster receives the templates rendered with the development values.

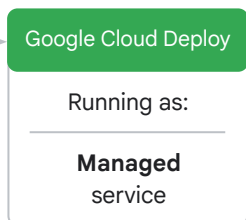
The delivery pipeline delivers the rendered manifests to the right targets

Define the
Scaffold runtime



create →

Manage the **pipeline**



clouddeploy.yaml

```
apiVersion: deploy.cloud.google.com/v1
kind: DeliveryPipeline
metadata:
  name: helloworld-pipeline
description: delivery pipeline
serialPipeline:
  stages:
    - targetId: dev
      profiles: []
    - targetId: staging
      profiles: []
    - targetId: prod
      profiles: []
```

The delivery pipeline is typically defined in a YAML configuration file, and represents the workflow to deliver an application from dev to staging to production.

Defining deployment progression

clouddeploy.yaml

```
apiVersion: deploy.cloud.google.com/v1
kind: DeliveryPipeline
metadata:
  name: helloworld-pipeline
  description: delivery pipeline
serialPipeline:
  stages:
    - targetId: dev
      profiles: []
    - targetId: staging
      profiles: []
    - targetId: prod
      profiles: []
```

Google Cloud

An important element of the delivery pipeline YAML file is the progression specification.

The progression connects the pipeline with deployment targets, and determines the order of the deployment process.

Targets must be defined in the same project and region as the delivery pipeline. But the runtimes the targets deploy to can be in different projects and regions.

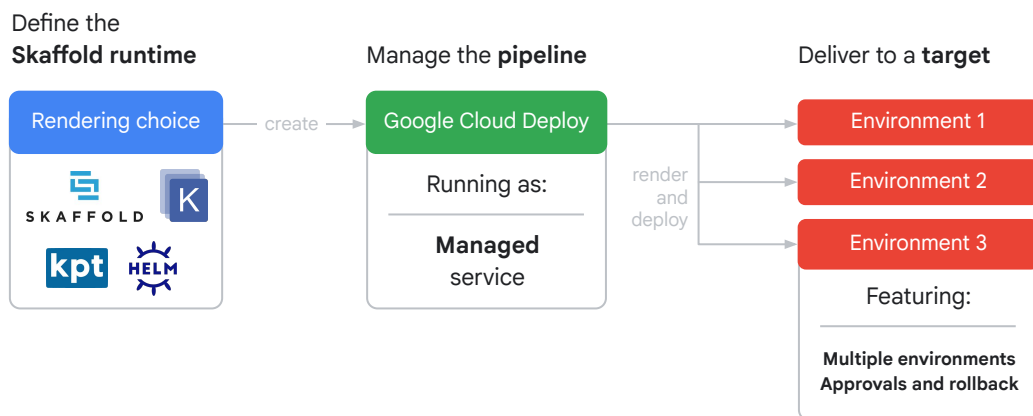
In Cloud Deploy there are serial and parallel progressions.

A serial progression is like a well-defined sequence of steps. Deployments move one after another in a specific order, as defined in the stages section. In this example, the deployment moves from the staging target to the prod target.

A parallel progression allows for simultaneous deployments to multiple targets. This can be useful for deploying to different test environments at the same time.

Within each stage, details are included like deploy parameters and verification steps.

The app is delivered to the correct target



Google Cloud

The final step is to deliver the application to the correct target environment, which could be a Kubernetes cluster, Cloud Run service, or other supported runtime. In this example the target is a GKE cluster.

Targets are defined in a delivery pipeline configuration file or a separate YAML file.

You can define targets and specify whether the rollout requires manual approval in a delivery pipeline configuration file, or in a separate YAML file.

Targets must be defined in the same project and region as the delivery pipeline, but the runtimes the targets deploy to can be in different projects and regions.

In Cloud Deploy you can roll back to a previous version. This is useful if a mistake occurred or to promote release to the next environment, like from staging to production.

CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security**
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network
- 08 Securing the software supply chain



Cloud Deploy is a tool with a wide range of features and capabilities. Let's look at how to configure policies, and deployments. We will also look at how security is implemented in Cloud Deploy.

Defining deployment policies

deploypolicy.yaml

```
apiVersion: deploy.cloud.google.com/v1
description: Restrict all rollouts in the deploy-policy-pipeline resource for the next ten
years
kind: DeployPolicy
metadata:
  name: helloworld-deploy-policy
selectors:
- deliveryPipeline:
  id: 'helloworld-pipeline'
rules:
- rolloutRestriction:
  id: no-rollouts
  timeWindows:
  timeZone: America/New_York
  oneTimeWindows:
  - start: 2025-01-01 00:00
    end: 2035-01-01 24:00
```

Google Cloud

In the deployment policy YAML file, you can restrict rollouts to specific times.

This example targets a previous delivery pipeline. A rollout restriction rule is defined to restrict all rollouts in the helloworld-pipeline resource for the next ten years.

Overriding deployment policies

Error from restrictive policy:

```
ERROR:
(gcloud.deploy.releases.create) A
create-rollout attempt was blocked
by the "helloworld-deploy-policy"
policy. Target: "prod-target",
Delivery pipeline:
"helloworld-pipeline", policy rule:
"no-rollouts"
```

Override a policy:

```
gcloud deploy releases promote \
--release=test-release-001 \
--project=PROJECT_ID \
--region=us-central1 \
--delivery-pipeline=helloworld-pipeline \
--to-target=prod-target \
--override-deploy-policies=helloworld-dep
loy-policy
```

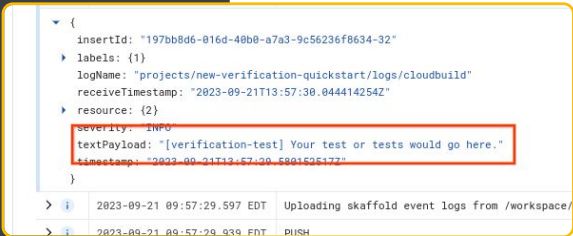
If a policy is too strict, it may result in an error output. You can override policies by adding the `--override-deploy-policies` flag when promoting a release.

Users must be granted the roles/clouddeploy.policyAdmin role for the override common to work.

This override ignores the deploy policy and creates the rollout to the prod-target.

Verifying deployments

```
apiVersion: skaffold/v4beta7
kind: Config
manifests:
  rawYaml:
    - k8s-pod.yaml
deploy:
  kubectl: {}
verify:
  - name: verification-test
    container:
      name: verification-echo
      image: ubuntu
      command: ["/bin/sh"]
      args: ["-c", 'echo "Your test or tests would go here."']
```



```
{
  insertId: "197bb8d6-016d-40b0-a7a3-9c56236f8634-32"
  labels: {}
  logName: "projects/new-verification-quickstart/logs/cloudbuild"
  receiveTimestamp: "2023-09-21T13:57:30.044414254Z"
  resource: {}
  severity: INFO
  textPayload: "[verification-test] Your test or tests would go here."
  timestamp: "2023-09-21T13:57:30.580163517Z"
}
```

2023-09-21 09:57:29.597 EDT	Uploading skaffold event logs from /workspace/
2023-09-21 09:57:29.939 EDT	PUSH

You can verify deployments in the skaffold.yaml configuration file, by creating an echo command to run after a successful deployment. The output appears under the textPayload log in that releases delivery pipeline details.

Deployments hooks

```
apiVersion: skaffold/v4beta7
kind: Config
manifests:
  rawYaml:
    - k8s-pod.yaml
deploy:
  kubectl: {}
customActions:
- name: predeploy-action
  containers:
  - name: predeploy-echo
    image: ubuntu
    command: ["/bin/sh"]
    args: ["-c", 'echo "this is a predeploy action"']
- name: postdeploy-action
  containers:
  - name: postdeploy-echo
    image: ubuntu
    command: ["/bin/sh"]
    args: ["-c", 'echo "this is a postdeploy action"']
```

```
apiVersion: deploy.cloud.google.com/v1
kind: DeliveryPipeline
metadata:
  name: deploy-hooks-demo-app-gke-1
description: main application pipeline
serialPipeline:
  stages:
  - targetId: hooks-staging
    profiles: []
    strategy:
      standard:
        predeploy:
          actions: ["predeploy-action"]
        postdeploy:
          actions: ["postdeploy-action"]
```

Google Cloud

Deployment hooks are also defined in the skaffold.yaml file. Deployment hooks allow you to execute actions before and after application deployments for greater control and automation.

These actions are configured to run at specific stages of the deployment process, providing flexibility for tasks like pre-deployment validation or post-deployment integration testing.

In your Cloud Deploy delivery pipeline, reference the skaffold.yaml configurations to specify when hooks should run (pre-deploy or post-deploy).

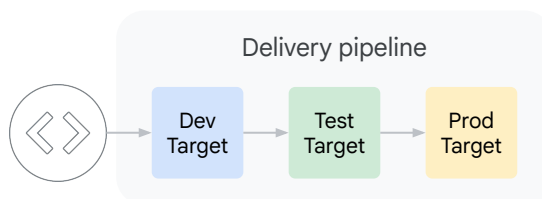
Cloud Deploy executes hooks within the designated environment, which can be the application's GKE cluster.

Promoting or abandoning releases

```
gcloud deploy releases promote \  
--delivery-pipeline web-app \  
--release web-app-001
```

```
gcloud deploy rollouts approve \  
web-app-001-to-prod-0001 \  
--delivery-pipeline web-app \  
--release web-app-001
```

```
gcloud deploy releases abandon \  
web-app-001 \  
--delivery-pipeline=web-app \  
--region=us-central1
```



In a delivery pipeline, successful releases are promoted to the next target in the pipeline, and unsuccessful releases are abandoned. For example a release might be promoted to a testing environment to ensure that the application functions properly. If successful, it's promoted to prod, and if unsuccessful, it's abandoned.

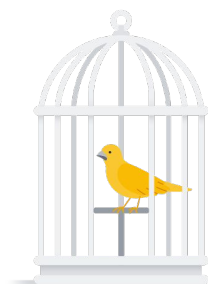
You can use the `gcloud deploy releases promote` command to promote a release and advance it by a single stage in the pipeline.

When a release is promoted to the final stage of the pipeline, approval is required to ensure untested code is not released to production. Approval is granted using the `gcloud deploy rollouts approve` command.

If a release fails the required tests, use the `gcloud deploy releases abandon` command to abandon the release. New rollouts can't be created from abandoned releases, and they can't be rolled back to.

Existing rollouts of abandoned releases are unaffected.

Cloud Deploy and canary deployments



Gradually move traffic to the new version of your application.



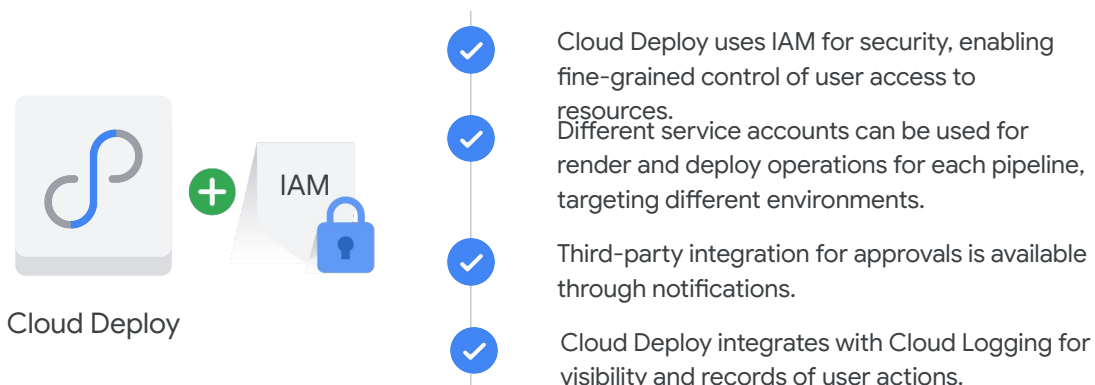
Cloud Deploy uses Cloud Service Mesh or Istio to split traffic by directing a specified percentage to the subset of users.

A common deployment strategy in Cloud Deploy is the canary strategy, in which new versions are rolled out to users gradually.

A small subset of users receives the new version, while the majority of users continue to use the stable, existing version.

Cloud Deploy uses service mesh technologies like Cloud Service Mesh or Istio to split traffic by directing a specified percentage to the subset of users.

Cloud Deploy uses IAM for security



Cloud Deploy uses IAM for security, enabling fine-grained control of user access to resource levels. This allows management to define which users can create, promote, and approve releases.

You can use different service accounts for render and deploy operations for each pipeline. These service accounts can target different environments and limit access.

Third-party integration support is available through notifications, enabling you to manage approvals from inside your existing tool or other ticketing solutions.

And Cloud Deploy integrates with Cloud Logging, for full visibility and records of user actions.

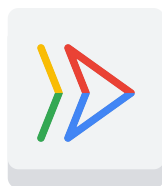
CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving**
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network
- 08 Securing the software supply chain



Besides Cloud Deploy, many organizations use Cloud Run for containerized applications.

Cloud Run



Cloud Run

A serverless development platform for hybrid and multi-cloud environments



Write code in a language of your choice.



Deploy applications in seconds.



Abstracts away the complexity of underlying infrastructure.



Map your services to your own domains.



Scale up or down automatically and only pay for what you use.

Cloud Run is a serverless development platform for hybrid and multi-cloud environments.

You can write code in a language of your choice with your favorite dependencies and tools, and deploy applications in seconds.

Cloud Run abstracts away the complexity of underlying infrastructure, enabling you to easily develop locally, and build and deploy apps anywhere.

It includes a stable HTTPS endpoint, TLS termination is handled for you, and you can map your services to your own domains.

Cloud Run has the ability to scale up or down automatically. You only pay for what you use.

Knative Serving



Managed Knative
Serving



Knative Serving, but with Google's added value and support.



Deep integration with CSM, Cloud Monitoring, Cloud Logging, and Cloud IAM



Enhanced security features, multi-cluster management, and consistent policy enforcement.



Abstracts away the complexity of underlying infrastructure.

Say:

At enterprise level, Google offers a managed instance of Knative Serving, integrated with GKE.

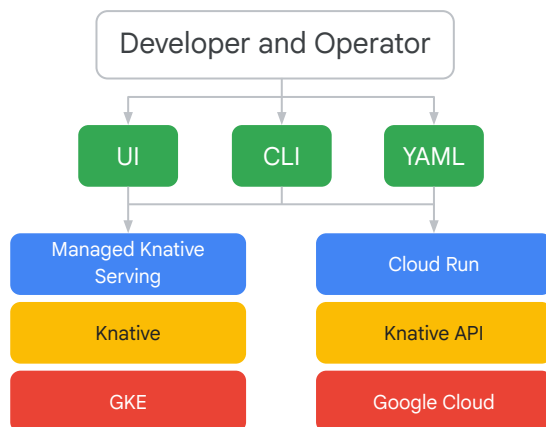
It is build on the open source Knative Serving, but with Google's added value and support. This managed experience simplifies installation, management, and updates.

It provides deep integration with Cloud Service Mesh for networking, security, observability , Cloud Monitoring, Cloud Logging, Cloud Identity and Access Management (IAM) for authentication using Workload Identity Federation.

Knative Serving supports enhanced security features like Binary Authorization, multi-cluster management through Fleets, and consistent policy enforcement using Policy Controller.

Like Cloud Run it abstracts away much of the underlying Kubernetes complexity.

Managed Knative Serving is serverless



Google's Knative Serving offering is serverless and container-based workloads can be deployed and managed in Google Cloud, hybrid, or multi-cloud environments.

Like Cloud Run, it uses a shared responsibility model. You or a partner are responsible for the underlying hardware infrastructure.

Differences between Cloud Run and Managed Knative Serving

	 Cloud Run	 Managed Knative Serving
Pricing	Pay per use	Included as part of GKE
Machine types	Limited CPU, GPU and memory	Standard or custom machine types including GPUs
Autoscaling	Up to 1000 container instances by default; quotas can be increased	Limited by the capacity of your GKE clusters
Identity and policy	IAM-based; VCP SC	RBAC controls; internet, cluster, or VPC accessibility
Networking	Access to VPC via Serverless VPC Access	Access to VPC; Services participate in the Cloud Service Mesh
URLs	Automatic service URLs and SSL certificates	Custom domains only with manual SSL certificates
Container isolation	Strict container isolation with gVisor sandbox	Default Kubernetes container isolation
Execution environments	Fully managed on Google infrastructure	GKE clusters

Google Cloud

There are some differences between Cloud Run and Managed Knative Serving.

Cloud Run operates on Google Cloud infrastructure, so you pay each time you use the tool. CPU, GPU, memory, and the number of instances are limited.

Managed Knative Serving runs on your infrastructure enabling you to use your hardware and benefit from additional flexibility, like higher CPU and memory. You can even use GPUs if they're available. The amount of instances depends on the size of your GKE cluster.

Cloud Run uses IAM and other Google Cloud tools like VPC service controls and serverless VPC access for security.

Managed Knative Serving uses RBAC controls and Istio Network Policies. Access can be configured to allow communication with the internet, in the cluster, or in the VPC.

Because Cloud Run services are part of Cloud Service Mesh, communication with other mesh services can be controlled in a similar way.

Other Cloud Run features like custom domains, SSL certificates, or container isolation with gVisor sandbox are configured manually in GKE clusters.

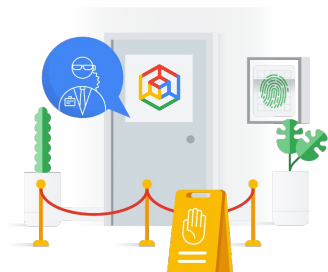
CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving**
- 07 CI/CD in a private network
- 08 Securing the software supply chain



While Cloud Run can run as services or jobs.
Knative Serving can only run as services.

Knative Serving services



As a service

A Knative Serving service:

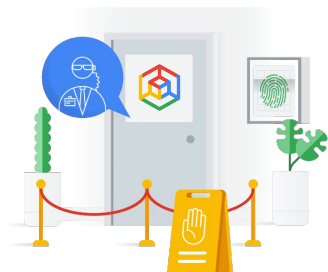
- Handles HTTP requests.
- Scales automatically.
- Ideal for web applications, APIs, and microservices.

Knative Serving service is for applications that need to be continuously available to handle incoming requests. You can think of it as a serverless web application platform.

A Knative Serving service:

- Listens and responds to HTTP requests.
- Scales automatically based on traffic.
- Is ideal for web applications, APIs, microservices.

Knative Serving services



As a service

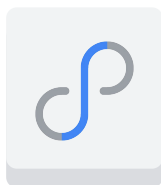
Considerations:

- Focuses on new revisions of your container image.
- Canary deployments and traffic splitting can minimize disruptions.
- Health checks are critical.

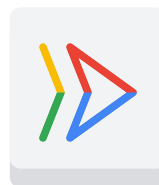
There are some Knative Serving service deployment implications to be aware of.

- Cloud Deploy focuses on updating your service with new revisions of your container image.
- Canary deployments and traffic splitting are often used to minimize disruptions.
- Health checks are critical to ensure Knative Serving can correctly scale and manage your service.

Cloud Run and Knative Serving



- Cloud Deploy manages new revision deployments.



- You have control over infrastructure, networking, and security.
- You are responsible for managing GKE clusters.
- Integrates with other Kubernetes workloads.

So, where does Cloud Deploy come into the equation?

For Knative Serving Services, Cloud Deploy manages the deployment of new revisions. This enables deployment strategies like canary or blue/green deployments, to smoothly transition traffic to the new version.

Remember that Knative Serving runs on your GKE clusters. This gives you control over the underlying infrastructure, networking, and security, but you are responsible for managing the GKE clusters.

Knative Serving integrates with other Kubernetes workloads. You can use Kubernetes services to access these services.

When to use a canary deployment?



Ashwin
Developer

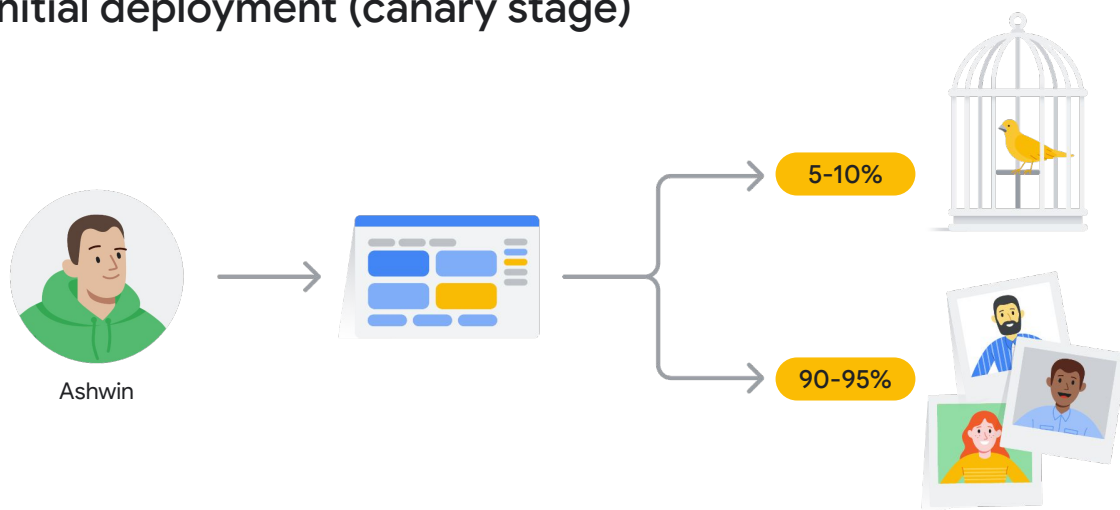
- ✓ Responsible for running a large e-commerce platform using GKE.
- ✓ Must ensure that the user authentication and authorization microservice doesn't experience any downtime or errors.
- ✓ Developed a new version with performance improvements and enhanced security and must minimize the risk of a widespread outage.

So when might you want to use a canary deployment with Knative Serving?

Meet Ashwin. Ashwin is responsible for deploying a new version of product recommendation engine for an ecommerce application. This engine is critical for driving sales, and bugs or performance issues in the new version could significantly impact revenue and customer experience.

Ashwin developed a new version of this authentication service with significant performance improvements and enhanced security features. However, he is also introducing potentially breaking changes to the authentication logic. Ashwin wants to minimize the risk of a widespread outage during the rollout.

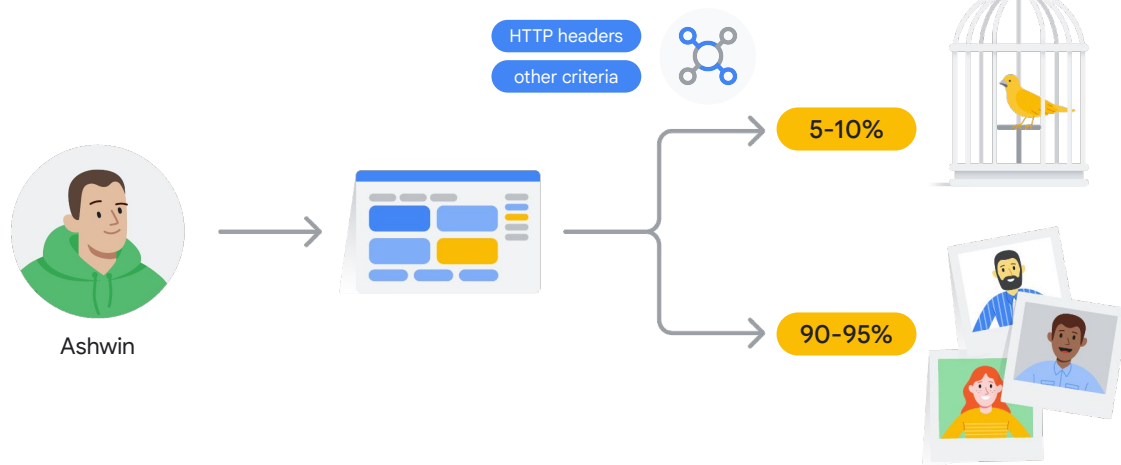
Initial deployment (canary stage)



Using Knative Serving's traffic splitting features, Ashwin plans to deploy the new version to a small subset, somewhere between 5-10%, of the app's production traffic.

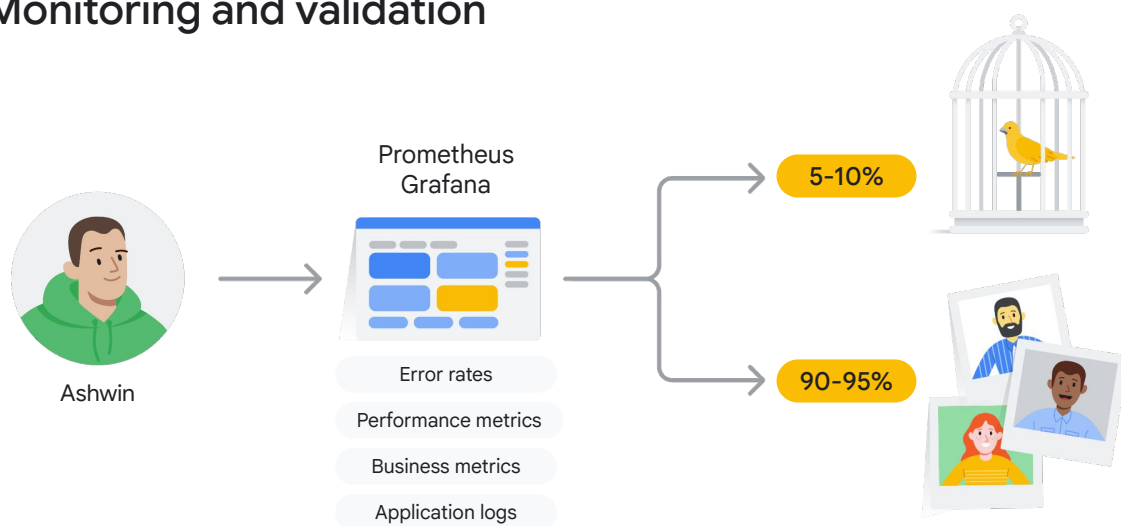
This subset of users, the canary group, experiences the new version of the service, while the majority of users continue to use the stable, existing version.

Cloud Service Mesh and Istio traffic management



Ashwin can use Cloud Service Mesh or Istio for more sophisticated traffic management, such as routing traffic based on HTTP headers or other criteria. This allows for very fine-grained control over the canary deployment.

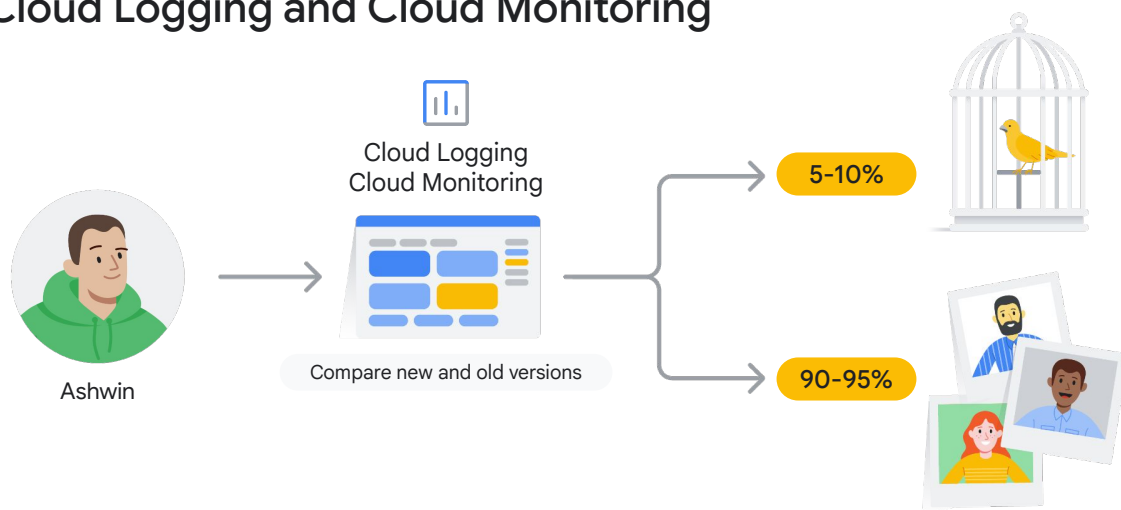
Monitoring and validation



Ashwin closely monitors the performance, error rates, and logs of both the new and old versions. He is looking for any anomalies in the new version.

Ashwin can use Prometheus and Grafana (common in GKE environments) to collect and visualize custom metrics from the authentication service, giving him deeper insights into its performance.

Cloud Logging and Cloud Monitoring



GKE provides centralized logging and monitoring across the entire environment. This allows Ashwin to easily compare the behavior of the new and old versions of the authentication service, even if other parts of the application are running on GKE clusters.

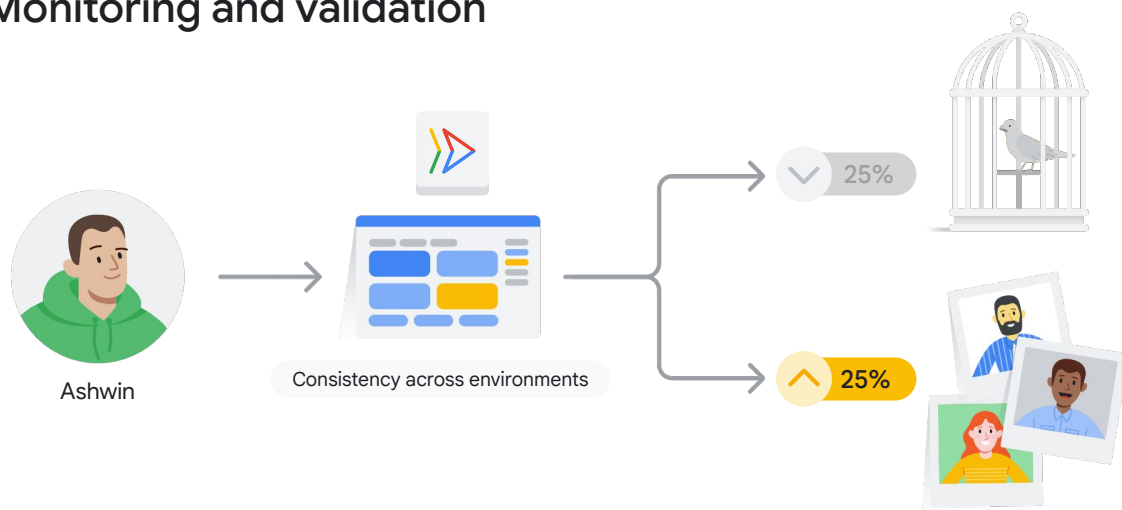
Monitoring and validation



If the monitoring and validation reveal no issues, Ashwin gradually increases the percentage of traffic directed to the new version using progressive rollout capabilities.

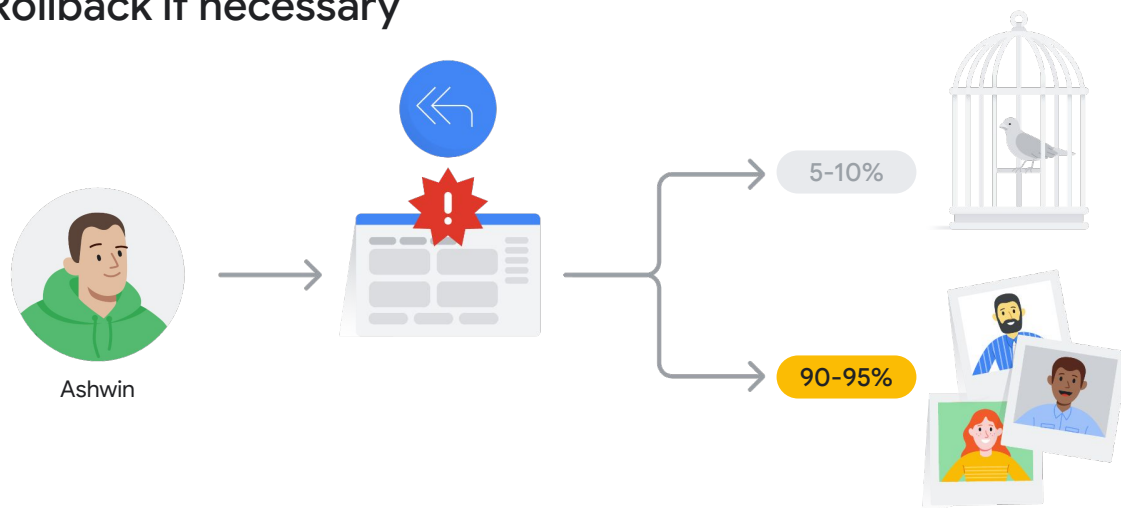
Ashwin increases the traffic in increments (e.g., 25%, 50%, 100%), allowing him to observe the impact at each stage.

Monitoring and validation



If there are GKE clusters running on-premises or in other clouds, Knative Serving allows you to use a consistent canary deployment strategy across Ashwin's entire GKE environment.

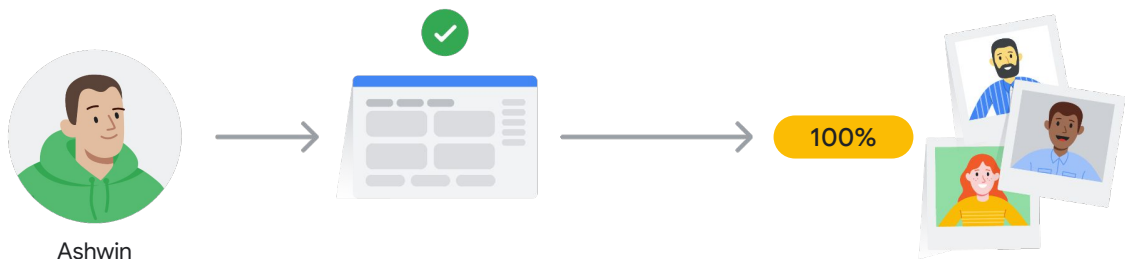
Rollback if necessary



If any problems are detected during the canary or progressive rollout, she can quickly and seamlessly rollback to the previous stable version.

This minimizes the impact on users and prevents widespread disruption.

Full deployment



The new version has been thoroughly evaluated and deemed stable, so Ashwin completes the rollout, directing 100% of traffic to the new service.

CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network**
- 08 Securing the software supply chain



Google Cloud

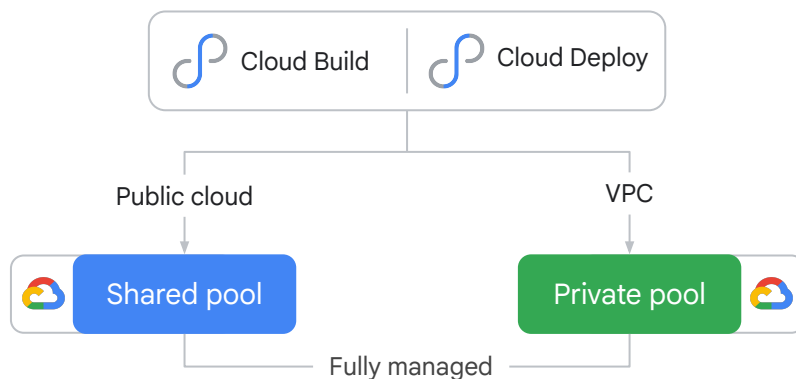
CI/CD is rarely an isolated process. CI/CD tools integrate with source code, artifact repositories, runtimes, and other services to automate developer workflows.

Most enterprise organizations have resources running in a private network, like in a VPC, on-premises, or on another cloud provider.

If it's behind a firewall, Cloud Build cannot securely access these resources unless access is created in the firewall, and it is advised that you do not take this approach.

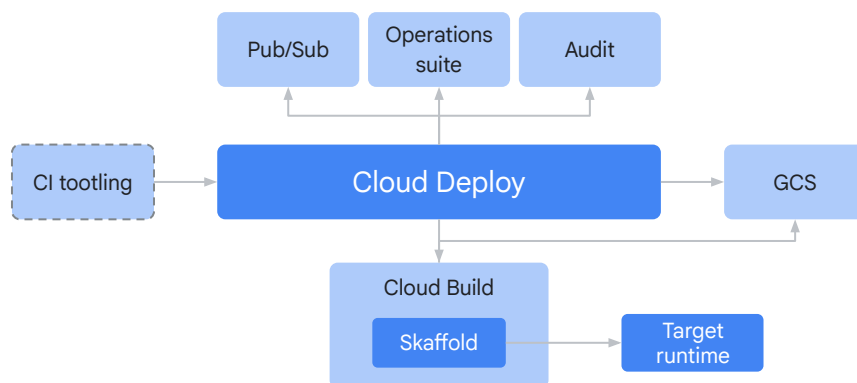
So how do you deploy CI/CD tools on a private network?

Shared and private pools



Cloud Build and Cloud Deploy offer two modes for running worker pools in Google Cloud: shared and private. Shared pools are global and can access the internet. Private pools are regional and run in a peered VPC network. Both are fully managed and share the same APIs.

Cloud Deploy uses the backend from Cloud Build for rendering and deployments

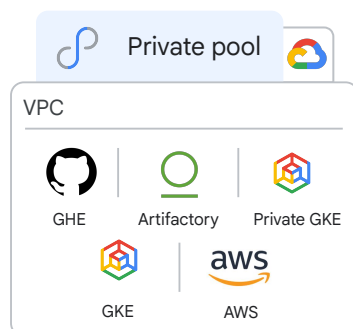


Cloud Deploy uses the backend from Cloud Build for rendering and deployments.

The user interface or CLI tools act as the control plane, but the actual execution occurs in Cloud Build. This is important because all features available to Cloud Build are applicable to Cloud Deploy.

CI/CD in a private pool connects with your VPC

Fully managed, customizable workers
peered into your VPC



Features	Shared Pool	Private Pool
VPC		X
VPC-SC		X
No public IP		X
Static IP ranges		X
Warm machine types	5	15
Max concurrency	30	100+
Regions	global	25

[Private pools](#) are controllable, scalable, and secure.

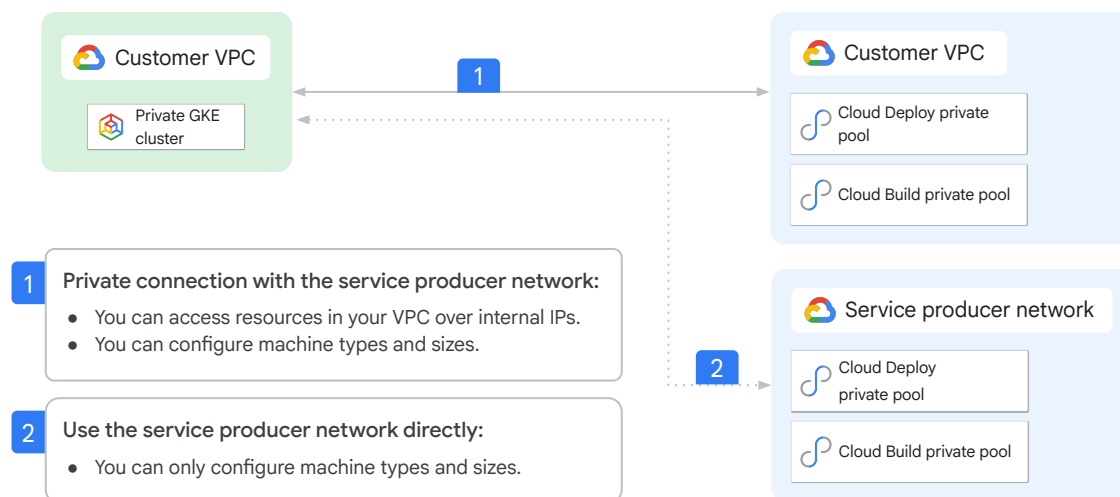
By placing the CI/CD worker pool within a peered VPC and implementing service perimeter controls, you restrict deployments to your clusters to a defined set of private IP addresses, enhancing security.

You can specify the types of machines that will perform the build process, keep a warm pool of machines to avoid long queue times, and scale to over 100 concurrent builds.

Finally, if you have data locality requirements, private pools are a great option because you can choose the country to run your CI/CD pipelines in.

Notice also that Cloud Build provides integrations with other platforms like GitHub Enterprise or Artifactory so you can continue using your existing tooling.

There are two options to configure private pools



Google Cloud

Let's examine how to configure private pools in Google Cloud.

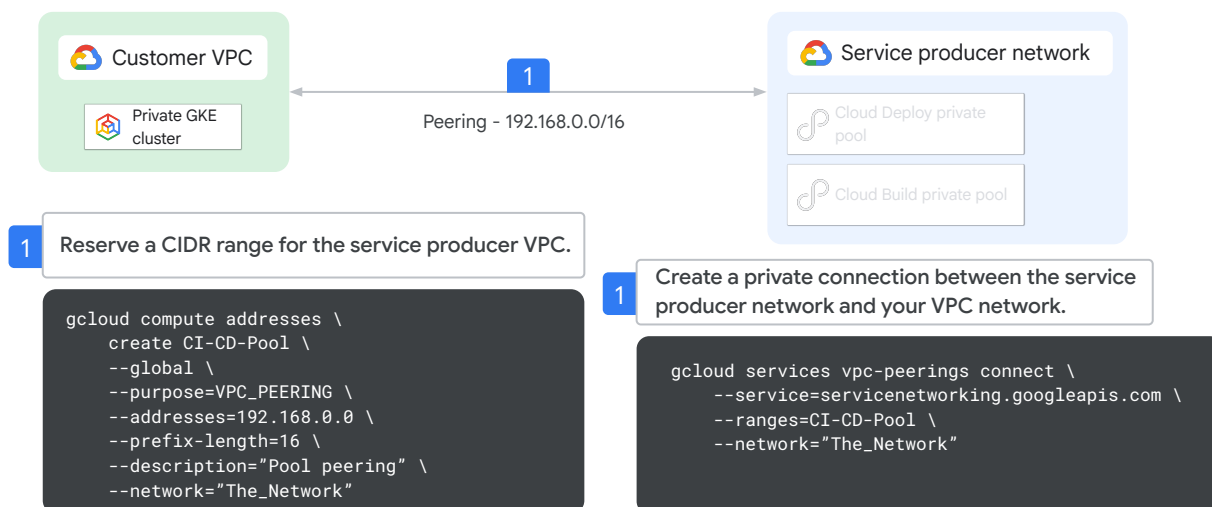
Private pools are hosted in a Google VPC network called the service producer network.

When configuring private pools, you can either use a private connection with the service producer network, or use the service producer network directly.

When using a private connection service producer network, you can peer your VPC with Google's network to communicate with your VPC resources over private IP addresses. If you choose not to peer your networks, the private pool will use a public endpoint to communicate with your VPC resources.

If you only need to configure machine types and sizes or deploy in a specific region, it's not necessary to configure the network, and you can use the service producer network directly.

1. Configure the network for a private pool

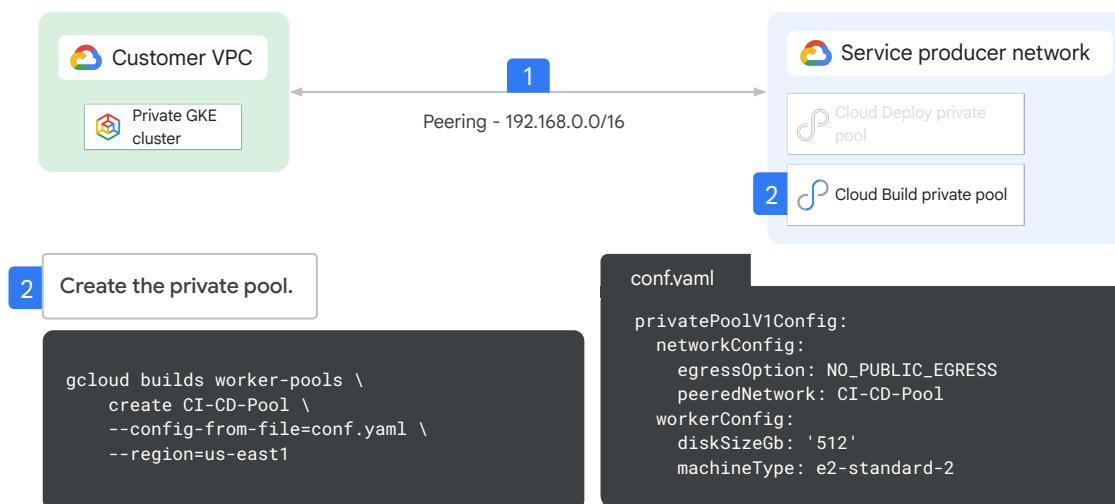


Google Cloud

To configure a private pool, the first step is to enable communication through private IP addresses by connecting networks.

For private GKE clusters, create a VPC network in your project. Then allocate a unique CIDR range for Google's service producer network, ensuring it doesn't conflict with your existing subnet ranges. Use the reserved CIDR range, create VPC peering between your VPC and Google's service producer network.

2. Create a new private pool

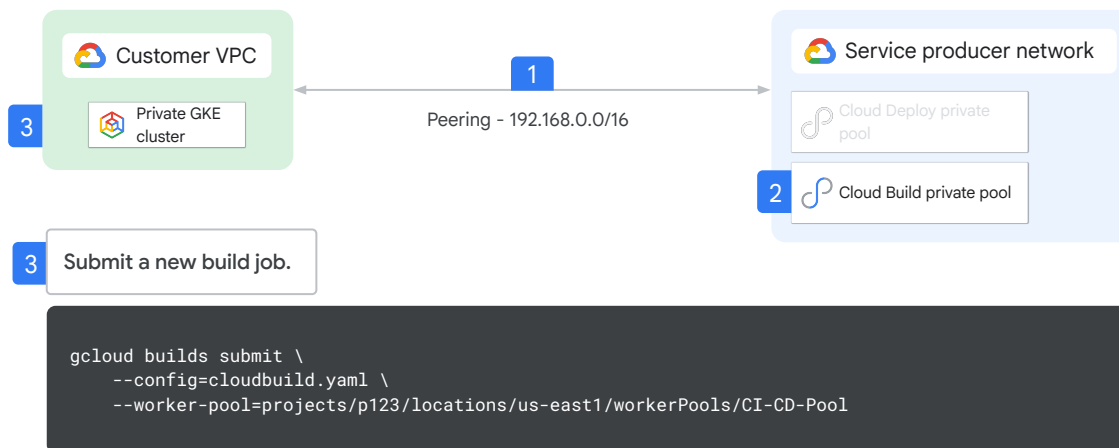


Google Cloud

Regardless of whether you created the peering, the next step is to create the private pool on Google's service producer network by creating a configuration file that specifies the type of egress, whether it should use a peered connection, and the configuration for the worker nodes, like the disk size and machine type.

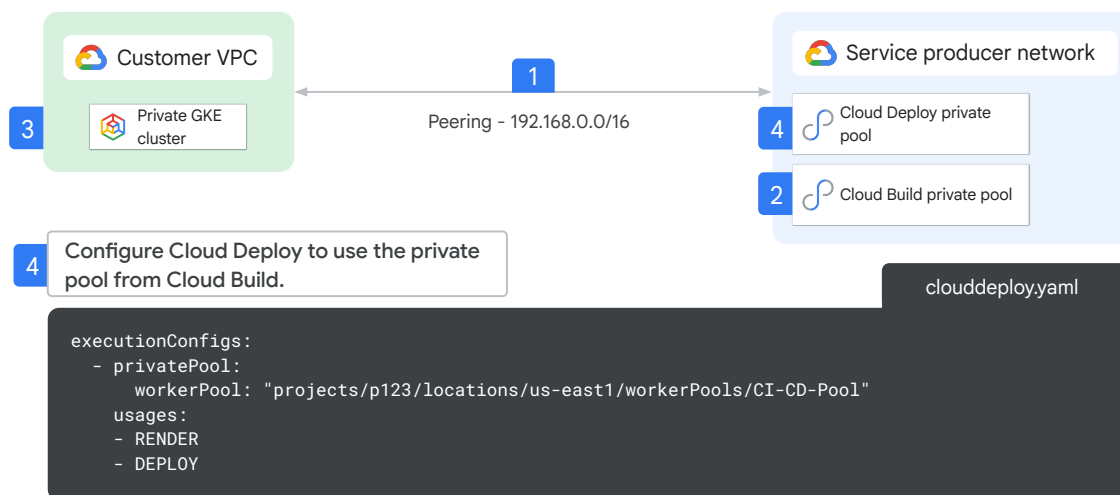
When the template is ready, run the "gcloud builds worker-pools create" command to build the private pool in the selected region.

2. Create a new private pool



Now you can start a new build with a Git trigger or by manually specifying the worker pool in a flag. You can monitor progress in the Google Cloud console.

2. Create a new private pool



Cloud Deploy runs within Cloud Build, allowing you to choose the worker pool it uses. There are two key configuration options. First, you can specify whether Cloud Deploy should use the same worker pool as your Cloud Build pipeline. Second, you can determine if the chosen pool applies to both the RENDER and DEPLOY stages, or only one of them.

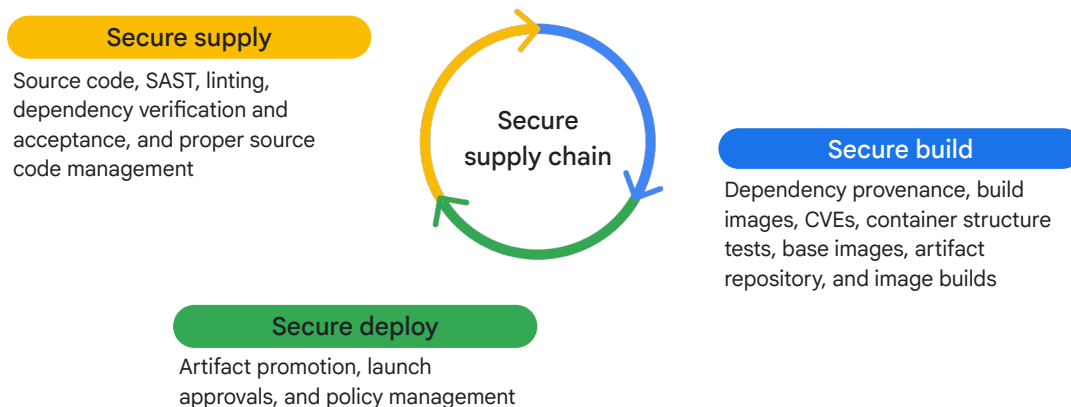
CI/CD at scale in GKE

- 01 CI/CD in Google Cloud
- 02 Cloud Build and GKE
- 03 Cloud Deploy and GKE
- 04 Cloud Deploy: Policies, Deployments and Security
- 05 Cloud Run and Knative Serving
- 06 Cloud Deploy and Knative Serving
- 07 CI/CD in a private network
- 08** Securing the software supply chain



So you learned how to build and deploy software into hybrid and multi-cloud environments. But how do you ensure that the software supply chain is secure?

Securing the software supply chain



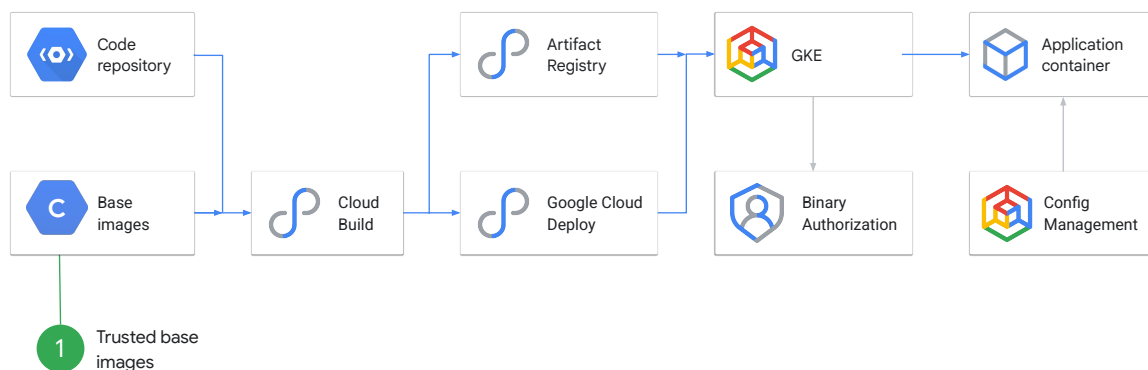
In regards to software, the supply chain consists of the supply, build, and deployment.

The supply that must be secured includes the source code, SAST, linting, dependency verification and acceptance, and proper source code management.

Securing the build includes trust in both the build tools and the output artifact. Verify dependency provenance, build images, CVEs, container structure tests, base images, artifact repository, and image builds.

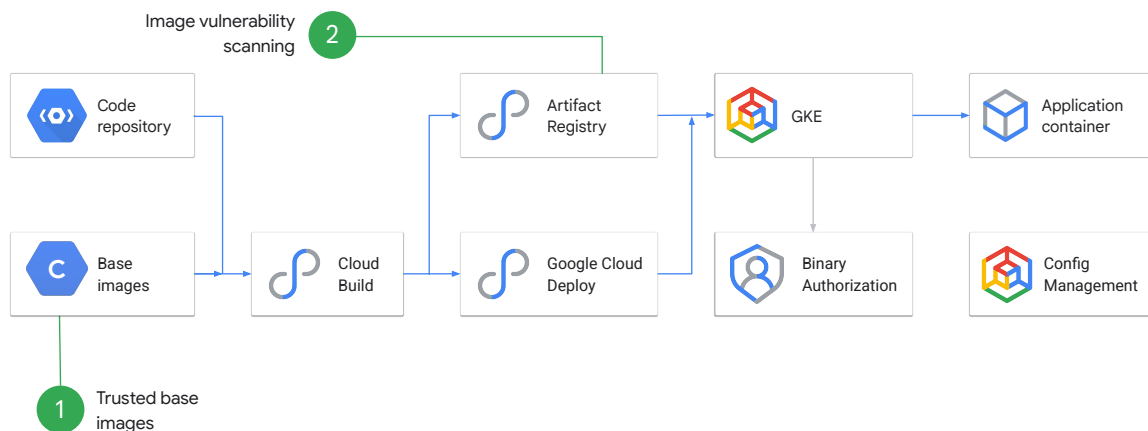
Securing the deployment includes the repeatable processes used to deploy securely to any environment: artifact promotion across the several stages, such as development or production, approvals of those launches, and policy management for the software deployed in the environments.

Deploy trusted base images



To secure the software supply chain, deploy trusted base images only. There are different ways to establish image trust. You can use base images managed by Google, distribution images, or create your own.

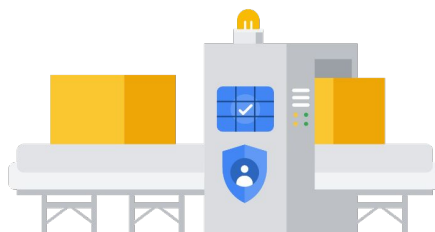
Enable the vulnerability scanning in Artifact Registry



You can use the vulnerability scanning feature in Artifact Registry to guard against vulnerabilities based on CVE feeds.

Artifact Registry offers a range of security features

- ✓ Achieve data locality with regional repositories
- ✓ Grant permissions at the repo level
- ✓ Use customer-managed encryption keys (CMEK)
- ✓ Secure with a VPC service perimeter
- ✓ Use audit logging for access controls



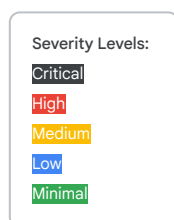
Google Cloud

Artifact Registry offers more than container storage; it provides a range of security features:

- Regional repositories reduce network egress and keep the CI/CD pipeline in the same region, thus achieving data locality.
- Permissions granted at the repo level create clear separation between accessing and publishing artifacts.
- Customer-managed encryption keys encrypt your data with your own keys.
- A VPC service perimeter adds an additional layer of security around your artifacts.
- Audit logging can be used to control who accesses artifacts.

Artifact Analysis and vulnerability scanning

Images are scanned when they are added to the registry or the vulnerability database is updated.



 Debian
  Ubuntu
  Alpine
  Red Hat
  CentOS

Digest details

f5fa01c635ca

gcr.io / bkauf-sandbox / gcpdemo @ sha256:f5fa01c635ca81a166c8614e4e3424b51bb8017e80edbc0677f26237a56b1483

Summary Vulnerabilities

Scan results

Scans performed: Alpine, Ubuntu, Debian

Total	Fixes	Critical	High	Medium
591	40	7	88	435

Filter by CVSS, package, or ID

Severity	CVSS	Fix available	Package	Documentation
Critical	10	—	tar	CVE-2005-2541
Critical	10	—	mercurial	CVE-2017-17458
Critical	9.3	Yes	glibc	CVE-2017-16997
Critical	10	—	systemd	CVE-2017-1000082

CVE-2005-2541

Vulnerability summary

Affected location: cpe:/o:debian:debian_linux:9

Package: tar

Version: 1.29b 1.1

Fixed in version: —

Vulnerability documentation

Debian: CVE-2005-2541

Google Cloud

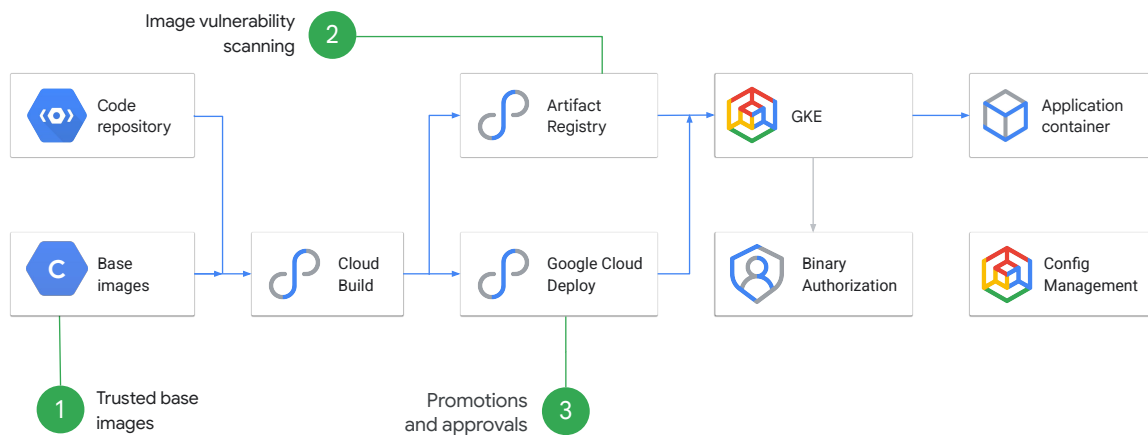
The Artifact Analysis API allows you to store metadata associated with a resource. This metadata can be later retrieved to audit your software supply chain.

When an image is stored in Artifact Registry, the registry automatically scans it for known common vulnerabilities and exposures, or CVEs. Artifact Registry examines images and packages installed within them, and works for many Distributions including Debian, Ubuntu, and Alpine, Red Hat and CentOS images.

Images are scanned when:

- They are added to the registry.
- The vulnerability database is updated.

Approve releases



IAM permissions allow you to create a clear separation of duties for deployments. Promoters receive permissions to indicate that artifacts are deployment-ready, while approvers, acting as stakeholders, receive permissions to control the actual live deployment timing.

Binary authorization



Ensures that the build goes through all stages of the pipeline, and that only properly signed containers are deployed to production.

Verify that:



Images are built by a trusted CI/CD pipeline.



Images pass security and quality tests like vulnerability scans or QA tests.



Only explicitly allowed third-party images are deployed.

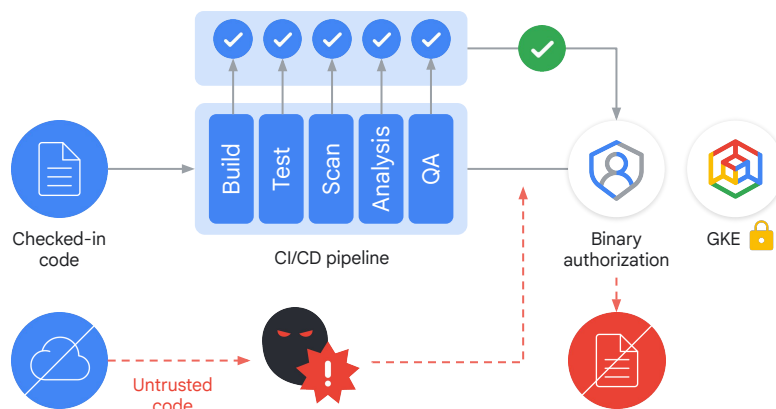
Binary Authorization ensures that the build goes through all stages of the pipeline, and that only properly signed containers are deployed to production.

Policy enforcement can be integrated directly into your clusters.

Binary Authorization supports signature verification and image allow lists, and can be used to verify that:

- Images are built by a trusted CI/CD pipeline.
- Images pass security and quality tests like vulnerability scans or QA tests.
- Only explicitly allowed third-party images are deployed.

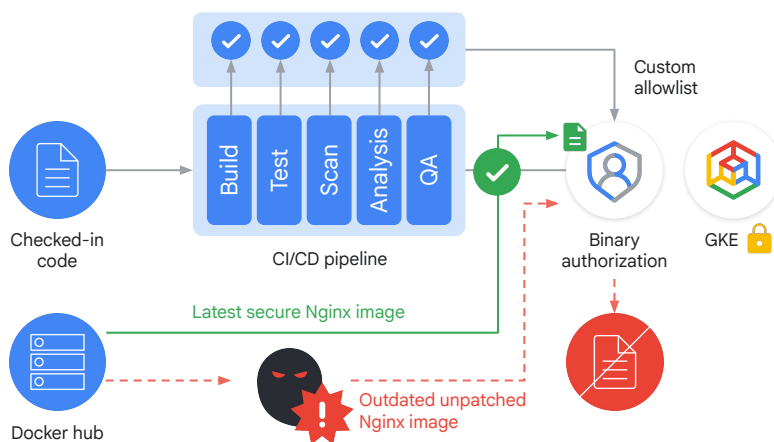
An image is attested to the desired CI/CD stages before deployment



Binary Authorization formalizes and codifies an organization's deployment requirements by integrating with the desired CI/CD stages to produce signatures, or "attestations," as an image passes through. Binary Authorization acts as an admission controller by preventing images that do not meet these criteria from being deployed. In Kubernetes Engine, this is implemented with the pod creation API, using the ImagePolicy admission controller.

Binary Authorization also integrates with [Cloud Audit Logging](#) to record failed pod creation attempts for later review.

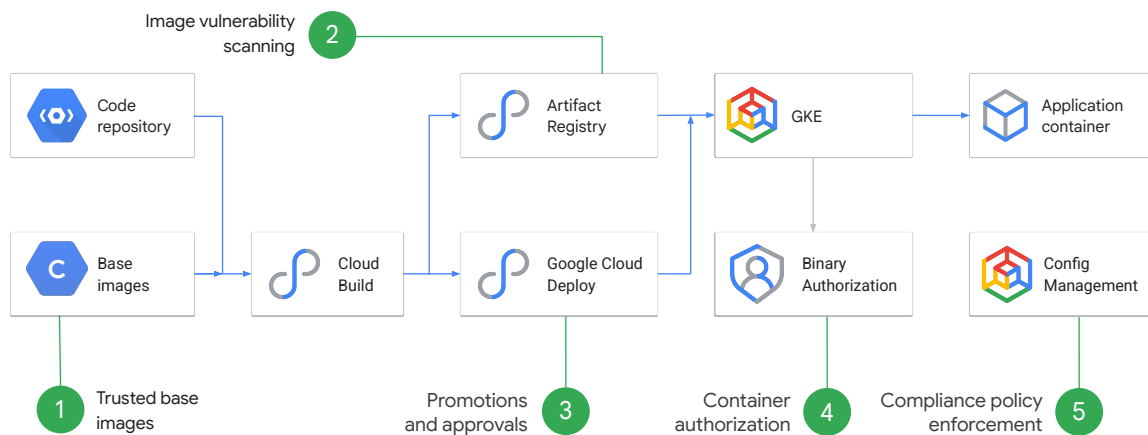
Allow only trusted third-party images to be deployed



A common source of production vulnerability is unpatched third-party software. These images are not built in-house, so they don't have the necessary attestations from a trusted CI/CD process.

This is why, in addition to signature-based verification, Binary Authorization policy also supports using name patterns to allowlist images. A user can specify a repository, path, or particular set of images that are allowed to deploy. You can have a centralized allowlist of all the third-party images and easily identify, locate, and update when a third-party image becomes outdated or vulnerable.

Approve releases



Finally, Config Sync enforces compliance policies. It uses an admission controller inside your cluster, which ultimately allows applications and configurations to be deployed in your cluster.